SVR INDIAN OIL SERVICE STATION

Daily Sales Report Data Entry System

SYSTEM DESIGN & ARCHITECTURE DOCUMENT

SDLC Phase: System Design & Architecture — follows Requirement Gathering (BRD)

Client	SVR Indian Oil Service Station Ponnur Pvt Ltd (IOCL Branch), Ponnur
Business Unit	SVR Indian Oil Service Station Ponnur Pvt Ltd
Document Type	System Design & Architecture Document (SDD)
Source Document	SVR-BRD-Requirement-Gathering.docx + SVR-BRD-Session-Update-2026-08-04.md (105 logged sessions, 2026-07-30 -> 2026-08-29)
Repository	https://github.com/Mithendra/IOCL_SVR
Version	1.0 — Design Baseline
Date	August 29, 2026
Prepared By	AI-assisted system design, from the confirmed BRD package (Claude)
Status	DRAFT FOR CLIENT & DEVELOPER REVIEW — see Section 19, Open Design Items

Table of Contents

(Right-click the table above and choose "Update Field" after opening in Word to populate page numbers.)

1. Introduction

1.1 Purpose of this Document

This System Design & Architecture Document (SDD) is the design-phase deliverable that follows the completed Business Requirement Document (BRD) for the SVR Indian Oil Service Station Daily Sales Report Data Entry System. The BRD — built up over 105 logged working sessions between July 30 and August 29, 2026, and validated against real station data (the AUG11 through AUG26 workbooks), 14 working HTML mockups, and physical paper forms — establishes what the system must do. This document establishes how it will be built: the component architecture, the data model, the calculation engine, the security model, the deployment model, and the specific design decisions and open questions that carry forward into development.

Every architectural choice below is grounded in a decision the client already confirmed in the BRD (technology stack, roles, module scope, business formulas). Where the BRD leaves a question open, this document does not silently resolve it — it is carried forward explicitly into Section 19, Open Design Items, with a recommendation, consistent with the discipline the BRD itself maintained throughout.

1.2 Scope

In scope: the desktop application architecture for a single-station deployment (SVR Ponnur), covering all 14 confirmed form modules, the Python/SQLite/ElectronJS technology stack, the OCR and Excel import/export pipelines, the calculation engine derived from the confirmed Daily Trial Balance formulas, the role-based access model, the Windows Service deployment model, and the CI/CD pipeline.

Out of scope: multi-station / multi-tenant architecture (the BRD confirms a single station, ~10 end users), a cloud-hosted backend (the confirmed architecture is a local desktop deployment), and mobile applications.

1.3 Inputs & References

- SVR-BRD-Requirement-Gathering.docx — the current-state Business Requirement Document (9 top-level sections, 39+ functional sub-sections)
- SVR-BRD-Session-Update-2026-08-04.md — 105-entry chronological session log documenting every requirement change, correction, and formula derivation from real data
- 14 branded HTML mockups (daily_sales_report, daily_sales_summary, daily_trial_balance, manage_users, rate_master, inventory_tracking, credit_remittance_master, new_credit_entry [retired], record_repayment [retired], monthly_expenses, employee_master, payment_receipt, yearly_sales_report, password_reset)
- Sample source data: GAS_STATION_AUG11_AUG12.xlsx and three SVR_Gas_Station_Daily_Sales_Report *_Filled workbooks, used throughout the BRD to derive and verify formulas
- SVR-Software-Install-Guide.pdf — 3-page install/troubleshooting guide already produced during the BRD phase

1.4 A Note on Business-Rule Volatility

Several figures in this system were corrected more than once during requirement gathering — most notably the Daily Trial Balance's testing/density deduction constant, which moved 10.0 → 10.5 → 21 (a documented misreading) → 10.5 → 10 (final, confirmed 2026-08-28), and the Section 6 Stock Value rate, which moved Buy Rate → Sell Rate → Buy Rate (three changes in one day, 2026-08-17). This is not noise to be smoothed over. Section 9 of this document proposes a specific architectural response — storing these figures as

versioned, database-backed parameters rather than hardcoded constants — precisely because this history shows they are the figures most likely to change again.

2. Architecture Overview

2.1 Architecture Style

The confirmed technology stack (BRD Section 4) is a single-station, offline-first desktop application: an ElectronJS frontend and a Python backend, both running on the station's own PC, sharing one local SQLite database file, with no server-side or cloud component. This is a layered desktop architecture, not a client-server web architecture — the “client” and “server” are two processes on the same machine, communicating over a local channel (IPC or a loopback HTTP API), and there is exactly one deployment target.

This style is a direct, deliberate fit to the confirmed scale: one station, ~10 total end users (1 Owner, 1 Manager, a handful of Pump Sales Men), a data volume of a handful of records per shift, and a 1-year retention window (BRD Section 4.3). The BRD's own Section 4.8 explicitly rejected over-engineering this system — e.g. choosing 3 Windows Services over 4, and dropping a 4-role model back down to 3 — and this document follows that same principle throughout.

2.2 System Context & Component Diagram

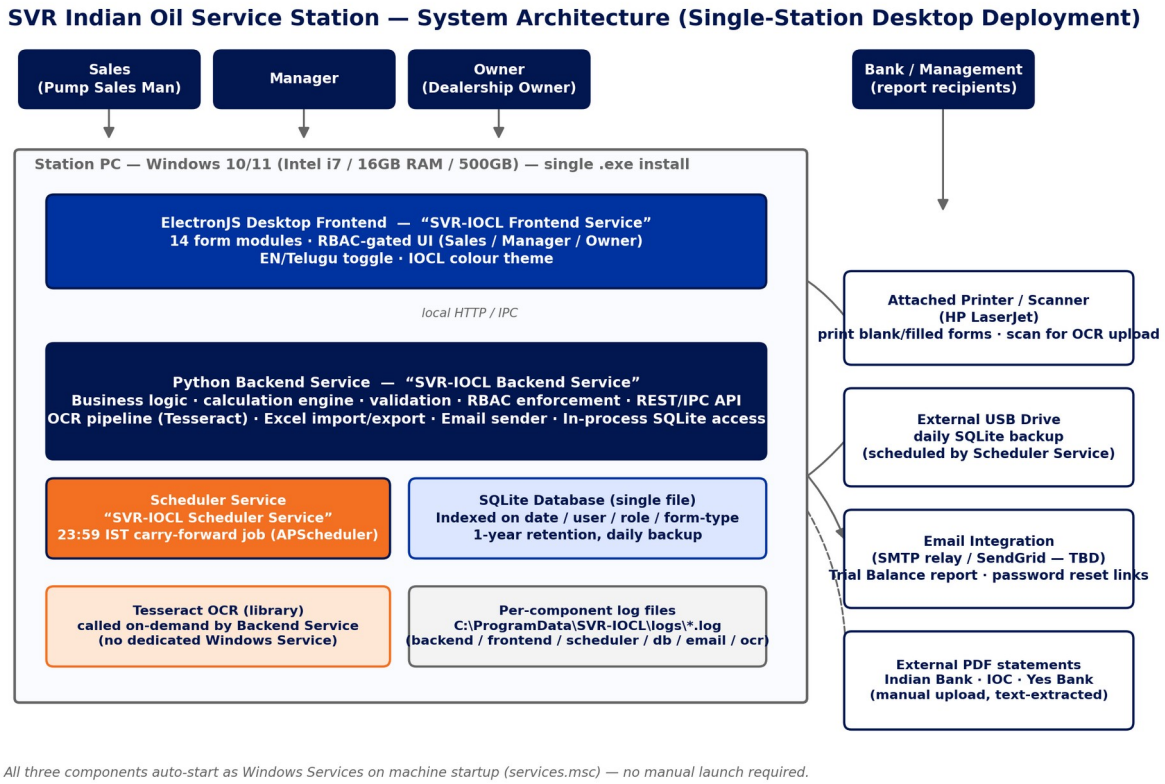


Figure 1 — High-level component architecture: actors, the three Windows Services running on the station PC, and the external devices/services they integrate with.

2.3 Component Responsibilities

Component	Responsibility
ElectronJS Frontend ("SVR-IOCL Frontend Service")	Renders the 14 form modules; enforces RBAC at the UI layer (hides/disables per role); language (EN/Telugu) and colour-theme toggles; triggers Print, Scan/Upload, Excel Import/Export, and Save actions against the backend.
Python Backend ("SVR-IOCL Backend Service")	All business logic and the calculation engine (Section 9); authoritative RBAC enforcement (never trust the client alone); the OCR pipeline (calls Tesseract on demand); Excel import/export; email sending; the only process that opens the SQLite file for writes; exposes a local API to the frontend.
Scheduler ("SVR-IOCL Scheduler Service")	Runs the 23:59 IST (Asia/Kolkata) carry-forward job (APScheduler) that copies each pump's Current Reading into the next day's Last Shift Reading, across Daily Sales Entry and Trial Balance; performs the catch-up-on-startup check for a missed rollover; runs the daily SQLite backup job to the external drive.
Tesseract OCR	A library called on demand by the Backend Service when a user uses Scan/Upload — not a standalone process or Windows Service, since it has no need to run continuously.
SQLite database	Single-file, indexed store for all application data; the sole source of truth; accessed in-process only by the Backend Service.

3. Technology Stack

Every row below is a confirmed BRD decision (Section 4 and its sub-sections), not a design-phase proposal — this table restates them as the design baseline.

Layer	Technology	Design Notes
Frontend	ElectronJS (desktop UI framework)	14 form modules; RBAC-gated UI; EN/Telugu toggle; 3-swatch IOCL colour theme
Application / business logic	Python 3.x	Validation, the calculation engine, RBAC enforcement, local API to the frontend
Database	SQLite (single file)	Indexed on date / user-role / form-type columns; 1-year retention; daily backup to external drive
OCR engine	Tesseract OCR (open-source)	Bundled into the installer; human-review-before-save required (accuracy caveat on handwriting)
Scheduler	APScheduler (Python), inside the Scheduler Service	23:59 IST (Asia/Kolkata, explicit) carry-forward job; catch-up-on-startup for missed rollovers
Email integration	SMTP relay or SendGrid (provider TBD — design-phase choice)	Daily Trial Balance report email; password-reset links
Packaging	electron-builder → single Windows .exe installer	Bundles Python + SQLite + ElectronJS + Tesseract; no separate prerequisite install on the station PC
OS process model	Windows Services (services.msc)	3 services — Backend, Frontend, Scheduler — Automatic startup; no manual launch required
CI/CD	GitHub Actions, windows-latest runner	lint → test → build → package → versioned release
Source control	GitHub — github.com/Mithendra/IOCL_SVR	Project-root CLAUDE.md for AI-assisted development conventions

Layer	Technology	Design Notes
Testing	pytest (backend) + Playwright (UI)	Recommended default — matches the tooling already used to validate all 14 mockups against real data
Target hardware	Intel i7 or better, 16GB RAM, 500GB storage	Attached printer/scanner (HP LaserJet); external USB drive for daily backups; up to 4 physical pumps / 2 dispensing machines

4. Roles & Access Control Model

4.1 Role Definitions

The BRD's role model changed twice on 2026-08-29 (Section 2.1) and the second change is CONFIRMED FINAL: three roles — Sales, Manager, Owner — replacing the original four-role table (Manager / Administrator / Super User / Data Entry) and a briefly-introduced fourth role (Read Only) that the client eliminated the same day, explicitly citing the application's small footprint (~10 end users) as the reason not to over-engineer access control. This document uses the final 3-role model as the design baseline.

Final Role	Supersedes	Description
Sales	Data Entry (Pump Sales Man)	Station-floor role. Access restricted to the Daily Sales Entry form only (confirmed 2026-08-16); no access to Inventory Tracking, Manage Users, Rate Master, Trial Balance, or Credit/Remittance Master.
Manager	Manager (unchanged)	Broad operational access: create/edit/delete entries, view reports, most modules — the only role permitted to delete an uploaded/submitted daily sales form. View-only on Rate Master.
Owner	Super User (the Dealership Owner)	All Manager privileges plus the exclusive right to edit Rate Master. Confirmed as the Dealership Owner specifically — not a general elevated-permission tier other staff can hold.

> *Open item: the original Administrator role (manage users, configure system, cannot delete forms) was not explicitly re-mapped when the role model was simplified to three. This document assumes Administrator's permissions fold into Manager, since Manager's original permissions were already a superset. Confirm with the client before development sign-off (Section 19, item 25).*

4.2 Role × Module Access Matrix

This matrix consolidates every access-control decision made across the BRD. Rows marked “not yet re-validated” were gated under the earlier 4-role naming and have not been explicitly re-tested against the final 3-role model — closing that gap is listed as an open item (Section 19, item 24).

Module	Sales	Manager	Owner	Notes
Daily Sales Entry	Create/edit own submission, print, scan/upload	Full, incl. delete	Full, incl. delete	Sales restricted to this form only (confirmed 2026-08-16)
Daily Sales Summary	Submit/verify own pump's data	Full	Full	Upload blocked until both pumps verified
Daily Trial Balance	No access	Full	Full	Not yet re-validated against 3-role model
Rate Master	Blocked	View-only	Full edit	3-tier gate implemented & tested (2026-08-29)

Module	Sales	Manager	Owner	Notes
Inventory Tracking	No access	Full	Full	Confirmed 2026-08-16
Credit / Remittance Master	Blocked	Full	Full	Functional gate implemented & tested (2026-08-28)
Employee Master	Blocked	Full	Full	Functional gate implemented & tested (2026-08-28); sensitive bank data
Monthly Expenses	No access (inferred)	Full	Full	Not yet explicitly re-validated
Manage Users	No access	Full	Full	Legacy: Manager/Administrator/Super User only
Payment Receipt	Open — likely usable at point of sale	Full	Full	Role gating not yet defined for this module
Yearly Sales Report	No access (inferred)	Full	Full	Not yet explicitly re-validated
Password Reset	Self-service, own account	Self-service, own account	Self-service, own account	Admin-initiated reset also available to Manager/Owner via Manage Users
New Credit Entry (retired)	—	—	—	Retired 2026-08-28: banner + disabled Save; kept for historical reference
Record Repayment (retired)	—	—	—	Retired 2026-08-28: banner + disabled Save; kept for historical reference

4.3 Access-Control Enforcement Design

The HTML mockups implement RBAC as a client-side “Logged in as” demo selector — this is explicitly documented throughout the BRD as an honest stand-in for real authentication, not the production design. Two concrete lessons from the mockups carry forward directly:

- Defense in depth (Employee Master, Section 5.33): the UI hides restricted content for a blocked role, and independently, the Save action itself refuses if the active session's role is not permitted — the same double-check pattern must exist in the real system, with the second check enforced server-side (Backend Service), not just re-implemented in the Electron renderer.
- Enforcement must move server-side entirely: the real system authenticates a session once at login (Section 13.1) and the Backend Service checks the session's role on every write, independent of what the frontend renders. A client-side-only check can be bypassed by a modified client, exactly as the BRD's own 2FA security note (Section 5.14) warns for TOTP verification.

5. Functional Modules → System Mapping

The BRD's design work produced 14 validated HTML mockups, each mapping to one production ElectronJS screen module. This table is the authoritative module inventory for development planning.

Module	Reference Mockup	BRD §	Purpose
Daily Sales Entry	daily_sales_report_branded.html	5.4	Per-pump, per-shift entry (up to 2/shift, variable — 1 on out-of-service days). Manual / OCR-Scan / Excel-Import modes, all converging on one record structure; live calculation engine.

Module	Reference Mockup	BRD §	Purpose
Daily Sales Summary	daily_sales_summary_branded.html	5.23–5.25	Combines the Off-pump and Road-pump submissions into one daily record; verification gate blocks upload to Trial Balance until both are verified.
Daily Trial Balance	daily_trial_balance_branded.html	5.8	11-section daily reconciliation: pump/computer readings, load/unload, cash value, stock value, bank/cash reconciliation, the 26-column historical ledger.
Rate Master	rate_master_branded.html	5.11	Owner-controlled Buy/Sell rates for Diesel, Petrol, and 5 oil SKUs; push model to Daily Sales Entry & Trial Balance; rate-change history log.
Inventory Tracking	inventory_tracking_branded.html	5.10	Current Stock Levels (live snapshot) + Restock Entry, for the same 5 oil SKUs used system-wide; closes the Opening/Closing Stock loop with Daily Sales Entry.
Credit / Remittance Master	credit_remittance_master_branded.html	5.17	New Credit, Remittance, and a computed Creditor Balance Summary; supersedes the two retired forms below.
Manage Users	manage_users_branded.html	5.3	User CRUD, role assignment, admin-initiated password reset, per-user 2FA enable/disable.
Password Reset	password_reset_branded.html	5.1 / 5.3	Landing page for the emailed reset link; set/confirm new password with live match-checking.
Monthly Expenses	monthly_expenses_branded.html	5.15 / 5.18 / 5.19 / 5.39	Payroll + operational expense ledger; extensible category list; date-range and category-filtered reporting.
Employee Master	employee_master_branded.html	5.16 / 5.18 / 5.19	HR record (incl. sensitive bank details), Payroll Run, Accidental & Health Insurance, Pay Stub requests, date-range report.
Payment Receipt	payment_receipt_branded.html	5.20	Customer-facing, narrow point-of-sale receipt layout — the only per-transaction (not daily-total) artifact in the system.
Yearly Sales Report	yearly_sales_report_branded.html	5.28	Financial-year (Apr–Mar) tax-oriented summary; explicit “not tax advice, route through the CA” disclaimer.
New Credit Entry (RETIRED)	new_credit_entry_branded.html	5.9 / 5.35	Retired 2026-08-28 in favour of Credit/Remittance Master; file retained, Save disabled, red banner shown.
Record Repayment (RETIRED)	record_repayment_branded.html	5.9 / 5.35	Retired 2026-08-28 in favour of Credit/Remittance Master; file retained, Save disabled, red banner shown.

6. Frontend Architecture (ElectronJS)

6.1 Navigation & Screen Map

The Home Page (BRD Section 5.2) branded “SVR Indian Oil Service Station Ponnur Pvt Ltd” is the landing screen, navigating into the 12 active modules of Section 5 above (the 2 retired forms remain reachable only for historical lookup, not from primary navigation). Each screen is RBAC-gated per the matrix in Section 4.2 — a screen a role cannot access is not merely disabled but not rendered/navigable, consistent with the “genuinely hides content” pattern already validated on Employee Master and Credit/Remittance Master.

6.2 UI / Branding System

- Colour palette: the three verified official IOCL logo colours — Vivid Tangelo Orange #F37022, Oxford Blue #02164F/#0033A0, White #FEFEFE — plus a Red #e31e24 legacy accent kept as a user-selectable 3-swatch theme (Section 5.12). Implemented as a single CSS custom property (--io-accent) driving every themed element — carry this pattern directly into the Electron renderer's design tokens.
- Language: an English/Telugu pill toggle, static pre-translated labels (no external translation service), scoped to section headings only per the confirmed design.
- Layout: A4 Landscape is the final, production-confirmed print orientation for Daily Sales Entry (reconfirmed 2026-08-28 after a same-day Portrait/Landscape investigation — see Section 16.5 for the full reasoning and the lesson it produced about trusting working code over stale documentation).

6.3 Print, Import & Export Architecture

Every form carries a consistent action set, generalised from the pattern built across all 14 mockups: Print (the current filled state), Print Blank Form (location/pump-specific pre-fill where applicable — Daily Sales Entry has two, split by physical pump), Scan/Upload (OCR entry point), Import from Excel, and Export to Excel. On Daily Sales Entry these four sit in one action bar scoped explicitly to Sections 1–7 (the auto-computed Daily Summary, Section 8, has nothing to import/export independently).

6.4 Client-Side Validation & Calculation Mirroring

The mockups' calculation engine (calcAll() and equivalents) must be mirrored, not just referenced, in the Electron renderer for responsive UX — but the backend recomputes and is the authority on every save (Section 7.3). Known UI-layer lessons already discovered and worth carrying into the real build directly:

- Guard against blank-field artifacts: a Consumption/Amount field must stay blank until its required inputs are actually entered, not compute a nonsensical value from an empty field defaulting to 0 (the exact bug found and fixed in Daily Sales Entry's Print Blank Form flow, Section 5.4.6 log).
- Free-text / expression-capable Amount fields: the Expenses section must accept an inline sum expression (e.g. “527+588+100=1215”), matching how pump sales men actually write on the paper form (BRD Section 8).
- All repeating-row sections must submit successfully with zero rows — several real paper samples show entire sections left blank on shifts with no such activity.

7. Backend Architecture (Python)

7.1 Service Decomposition

Three Windows Services (BRD Section 4.8, finalised over an explicitly-rejected 4-service and 5-service alternative):

Service	Runs	Why it is its own service
SVR-IOCL Backend Service	Python business logic, calculation engine, validation, RBAC enforcement, OCR pipeline, Excel import/export, email sending, and all SQLite access (in-process)	The only continuously-running process that needs to be independently monitorable/restartable and that owns the database file

Service	Runs	Why it is its own service
SVR-IOCL Frontend Service	The ElectronJS desktop UI	Separated from Backend so the UI can restart independently of business logic; note — Electron apps are more commonly configured as user-session startup items than true Windows Services; confirm this during design (Section 19, item 23)
SVR-IOCL Scheduler Service	APScheduler running the 23:59 IST carry-forward job and the daily backup job	Runs unattended overnight; isolating it means a scheduler crash cannot take down the interactive Backend Service mid-shift

> *SQLite does not get its own service (it is a file, accessed in-process by the Backend Service, not a server process) and Tesseract does not get its own service (it is a library invoked on demand, not continuously running) — both were explicitly proposed and rejected during the BRD phase (Section 4.8).*

7.2 API / IPC Layer

The Frontend Service communicates with the Backend Service over a local API (Electron IPC or a loopback HTTP API bound to localhost only — a design-phase choice with no BRD preference stated; a loopback REST API is recommended here since it keeps the Backend Service testable independently of Electron, matching the pytest-first testing strategy in Section 17). Every write endpoint re-validates the session's role server-side per Section 4.3, independent of what the UI already enforced.

7.3 Calculation Engine

The full Daily Trial Balance formula set (Section 9 of this document) and the Daily Sales Entry calculation chain (gas/oil consumption × rate, section totals, Net Bal Hand-off) must be implemented once, server-side, as the single source of truth — with the frontend's own calculation code treated as a responsive-UX mirror, not the authority. This directly generalises the BRD's own confirmed rule for OCR and Excel-import data: “the system re-runs the same auto-calculation logic against the extracted values and flags any row where the read Amount does not match the recomputed value” (Section 5.4) — every entry path (manual, OCR, Excel import) must converge on this one engine.

7.4 OCR Pipeline (Tesseract)

Detailed in Section 11 below, alongside the full BRD Section 8 test-scenario library.

7.5 Import / Export Engine

- Excel export: confirmed sufficient as a structured data export — it does not need to visually replicate the paper form's layout (resolved 2026-08-29, BRD Section 4.8).
- Excel import: same auto-calculation and validation rules apply post-import as post-OCR — an imported Amount is recomputed and any mismatch flagged, never silently trusted.
- Section 10 of the Trial Balance (Daily Mgmt Summary) needs its own standalone export in addition to the full 11-section export — exact target format (single-row daily vs. running multi-day) is an open item (Section 19, item 4).

7.6 Email Integration

Required for two flows: the daily Trial Balance report to management, and password-reset / admin-initiated-reset links. Provider (SMTP relay vs. SendGrid vs. another transactional-email service) is not specified in the BRD and is a design-phase choice — recommend an SMTP relay for the MVP given the very low email volume (one report/day, occasional resets), avoiding a paid API-key dependency for a ~10-user system.

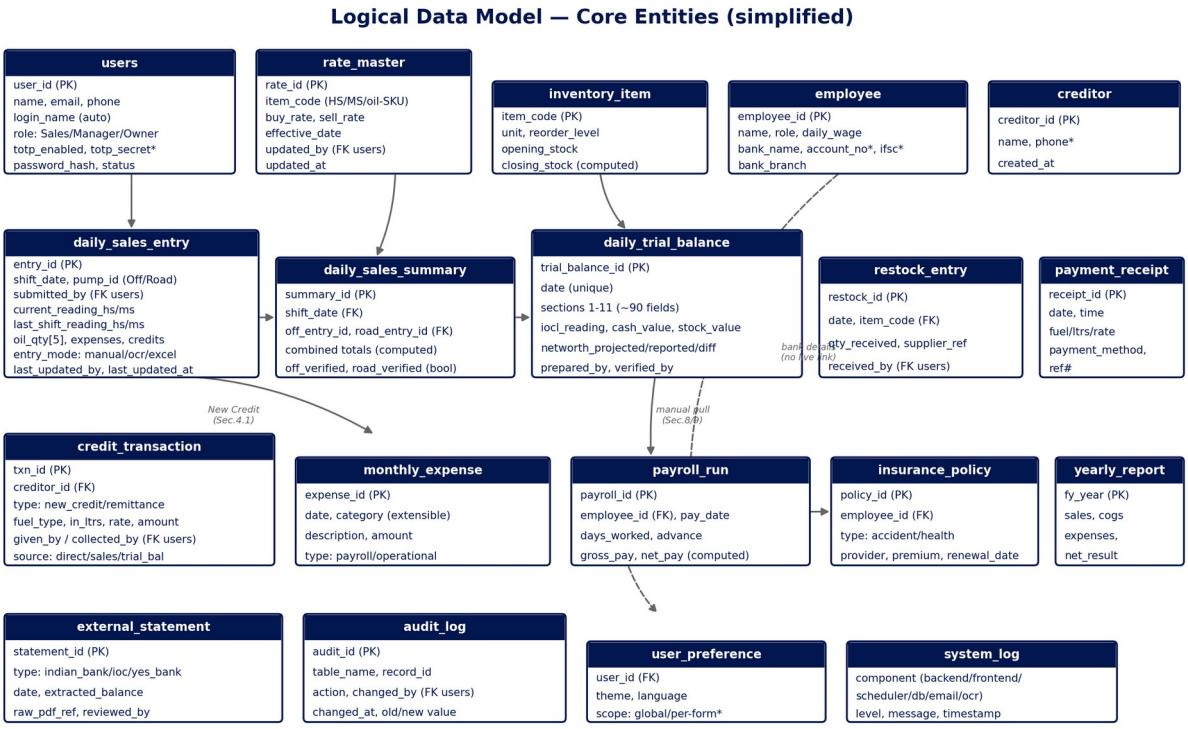
7.7 Scheduler & the 23:59 IST Carry-Forward Job

Confirmed feasible (BRD Section 5.4.1 / 40) as a standard APScheduler job inside an always-on Windows Service, explicitly using the Asia/Kolkata timezone (never the host machine's default). At 23:59 IST it reads each pump's Current Reading and writes it as the next day's Last Shift Reading directly in SQLite, for both Daily Sales Entry and Trial Balance (Sections 1, 2, 3, 5, 8 all carry-forward the same way). Three edge cases are confirmed as design requirements, not yet built:

- Missed rollover: if the PC is off at 23:59 IST, a catch-up-on-startup check must run the job on next launch (confirmed policy — BRD §9, item 7a).
- Gap days: a day with zero activity has no Current Reading to carry forward — the job must skip back to the most recent day that actually has a value, never assume literally “yesterday.”
- Post-rollover corrections: if a Current Reading is edited after the rollover already copied it forward, manual re-sync is acceptable — no auto-correction required (confirmed policy — BRD §9, item 7c).

8. Data Architecture

8.1 Logical Data Model



** sensitive field — encryption at rest + role-restricted access required (Account Number, IFSC, TOTP secret, phone numbers, external-statement PDFs)*

Every table carries last_updated_by / last_updated_at (Section 5.27) — omitted per-entity above for space; audit_log provides the full change history.

credit_remittance_master supersedes the retired new_credit_entry / record_repayment tables — both fold into credit_transaction.

Figure 2 — Simplified logical entity model. Dashed relationships are manual/cross-referenced today per the BRD, not live foreign-key-enforced links — see Section 8.5.

8.2 Core Entities

Entity / table group	Notes
users	System accounts. role ∈ {Sales, Manager, Owner}; login_name auto-derived (first initial + last name, editable on collision); totp_enabled/totp_secret for optional 2FA; password_hash — never a plaintext or displayed password anywhere, including admin-initiated resets.
rate_master	Buy/Sell Rate per fuel (HS/MS) and per oil SKU, with effective_date and updated_by — the versioning needed to answer the open rate-history question (Section 19, item 7).
inventory_item	One row per oil SKU (5 confirmed + operator-added items, Section 5.4.8/5.8.11); opening/closing stock computed from the Daily Sales Entry loop.
employee	HR master record, incl. sensitive bank fields flagged for encryption at rest (Section 13.3).
creditor	One row per named creditor; credit_transaction rows aggregate against it for the Creditor Balance Summary.
daily_sales_entry	One row per pump per shift per submitting Sales user (up to 2/day, variable); entry_mode records which of manual/OCR/Excel-import produced it.
daily_sales_summary	One row per shift_date, referencing both pump submissions; off_verified/road_verified gate the Trial Balance upload.
daily_trial_balance	One row per calendar date (unique); the ~90 fields across the 11 confirmed sections (Section 9's formula register) plus the 26-column historical ledger as a related table.
restock_entry / credit_transaction / monthly_expense / payroll_run / insurance_policy / payment_receipt / yearly_report	One table per module, each following the module's confirmed field list (Section 5).
external_statement	One row per uploaded Indian Bank / IOC / Yes Bank PDF statement, with the extracted balance and the reviewing user — see Section 12.
audit_log	Full change history — every write anywhere in the system logs table, record, action, actor, and old/new value, generalising the “Last Updated By/Date, no exceptions” requirement (BRD Section 5.27) that the client called non-negotiable.
user_preference / system_log	Per-user theme/language settings; per-component structured log entries mirroring the six log files in Section 14.

8.3 Indexing Strategy

BRD Section 4 confirms indexing is required to keep queries performant over the 1-year retention window, specifically calling out date, user/role, and form-type columns. Concretely: every daily-record table (daily_sales_entry, daily_trial_balance, daily_sales_summary, credit_transaction, monthly_expense, payroll_run) should carry an index on (date) at minimum, and a composite index on (date, submitted_by) where a report or RBAC check filters by both — e.g. the Date Range Report already validated on Employee Master and Monthly Expenses (Section 5.19), and the Credit Report's “pending payments first” sort (Section 6).

8.4 Data Retention, Backup & Archival

- Retention: 1 year maximum; records older than 1 year may be purged or archived off the primary system (confirmed, BRD Section 4.3).

- Backup: a daily SQLite backup to an attached external USB drive, run by the Scheduler Service alongside the 23:59 IST carry-forward job.
- Load-tested: the mockups' 26-column ledger was stress-tested at a full year's volume (365 rows, added in 0.32s, calculated in 0.008s, zero errors) and oil-item extensibility at 50 items — no performance concern surfaced at any realistic volume for this station's scale.

8.5 Data Normalization Strategy & Canonical Source-of-Truth

This is the single most consequential open architectural question in the whole system, and the BRD is explicit that it was raised, discussed, and deliberately deferred rather than resolved — it deserves to be stated plainly here rather than buried in the open-items list.

The Daily Trial Balance's own Sections 3, 8, and 9 duplicate data that Daily Sales Summary, Credit/Remittance Master, and Monthly Expenses also capture. A shared SQLite database does not, by itself, prevent the same real-world transaction from being typed twice into two different forms — sharing a database only prevents drift between copies, not duplicate manual entry in the first place. The recommendation given during the BRD phase (Section 5.8.8/5.8.9 of the log) was to designate one canonical entry point per data type — Credit Master owns credits/remittances, Monthly Expenses owns salaries/advances/operational expenses, Employee Master's Payroll Run owns bi-weekly payroll — with Trial Balance's matching sections converted to read-only, auto-pulled rollups. The client's decision at the time was to keep Trial Balance's fields manual, as already built, and record the recommendation for later reference rather than implement it.

This document restates that recommendation as ADR-1 (Section 18) rather than silently dropping it, because it directly determines two things a developer needs before writing a line of backend code: whether daily_trial_balance's Sections 3/8/9 are physical columns fed by manual INSERTs, or generated views/rollups computed from credit_transaction, monthly_expense, and payroll_run. Building the wrong one is expensive to unwind later. The design-phase recommendation is to confirm this decision explicitly with the client before backend development begins, even though it was deferred during requirement gathering.

9. Business Rule Engine — Daily Trial Balance Formula Register

The Daily Trial Balance is the most formula-dense module in the system — 11 sections, each depending on the ones before it, all independently re-derived and verified against real station data (the AUG11 through AUG26 workbooks) across many sessions. This table is the authoritative formula register for the calculation engine (Section 7.3), current as of the final BRD state (2026-08-29). It should be treated as the single specification developers implement against — not the mockup's JavaScript, which is a working reference implementation, not the record of truth.

§	Section	Confirmed Formula	Status
1	IOCL Computer & Pump Readings	Diff = Yesterday − Current (Yesterday auto-carried, 23:59 IST). Consumption pulled from Sec.3. Computer/Pump Diff = Consumption − Diff. Benefit/Loss = Consumption + Diff. Deduct Testing/Density = Consumption − 10, per fuel, always (even single-pump days).	Final, confirmed 2026-08-28
2	Load/Unload Details	Total = New Computer − Old Computer. Old Computer auto-carries from the last delivery. Applicable only when IOCL delivers a load.	Confirmed 2026-08-19
3	Day Sales Report (Oil Sales)	Consumption = Today − Last (Last auto-carries 23:59 IST). Amount = Consumption × Sell Rate. Oil Amount = Sold × Rate. Closing Stock = Opening − Sold. Daily Sales Total = fuel total + oil amounts.	Confirmed; cross-verified against real data on multiple days

§	Section	Confirmed Formula	Status
4	Daily Cash Value	4.5 = sum(4.1:4.4). 4.6 = 4.5. 4.12 = 4.6 + sum(4.7:4.11). 4.16 = 4.12 + sum(all creditor rows, variable count).	Confirmed. “Old Credit Cash hand-off” row confirmed INCLUDED despite its own “(Remove the Row)” annotation on the source sheet — flagged tension, not silently resolved
5	Book/Cash Value Reconciliation	5.1 auto-carries from prior day's 4.16. 5.3 = 5.1 + 5.2. 5.4 = pulled from 4.16. 5.5 = 5.4 − 5.3.	Confirmed
6	Stock Value	Ltrs auto-pulled from Sec.1 IOCL Current Reading. Rate = Buy Rate HS/MS (cost basis, not Sell Rate). Amount = Ltrs × Rate. Total = sum of both fuels.	Final 2026-08-25 — see volatility note below
7	Trial Balance (Total SVR Networth Reported)	7.1 read directly from Sec.5.4 (not independently recomputed). Total = 7.1 + Sec.6 Stock Value Total.	Confirmed
8	Trial Balance – IOCL Computer vs Pump	8.1 = prior day's own Sec.7 Total (auto-carried). 8.2 = pulled from Sec.9 Daily Profit. 8.3 (Projected) = 8.1 + 8.2. 8.4 (Difference) = today's Sec.7 Total − 8.3.	Confirmed. ±Rs100 threshold triggers a Supervisor/Manager → Owner phone call — the client's own stated workflow, not inferred
9	Daily Management Reporting	Expenses breakdown (Salaries Mid/End, Power Bill, Salary Advance) auto-summed. Total SVR Networth Value – Projected = Yesterday's + Daily Profit. Difference = Reported − Projected.	Confirmed structure. “Row 99” allowance formula (Daily Profit − ~Rs3,300 fixed daily allowance) verified only in the client's own Excel — NOT yet built into the digital form
10	Daily Mgr Calculation (26-column ledger)	One row per date, running log. “2T Sales” = the FULL 5-item Oil Sales subtotal, not just the two “2T”-named rows (corrected and verified against two real days' data).	Confirmed. Default row count reduced 30 → 5 (2026-08-25), reversing an earlier decision
11	Old/New Credit Sales Details / Fleet Card Balance Tracking	Minimal — only 2 rows observed in the one day of source data available.	UNDER REVIEW — scope not yet confirmed

9.1 Design Recommendation: Versioned Business-Rule Parameters

Two figures in the table above were each corrected multiple times during requirement gathering: the Section 1 testing/density deduction (10.0 → 10.5 → a mistaken 21 → 10.5 → 10, five states across three sessions) and the Section 6 Stock Value rate (Buy → Sell → Buy, three changes in a single day). Both are hardcoded constants in the current mockups. The recommendation for the real system is to store constants of this kind — the testing deduction, the ±Rs100 alert threshold, the ~Rs3,300 daily allowance once confirmed — in a database-backed system_parameter table (name, value, effective_date, updated_by), the same pattern already confirmed for Rate Master. This does not change any formula; it changes where the number that formula reads from lives, so the next correction is a data change an Owner/Manager makes through a UI, not a code deployment — directly addressing the pattern this project's own history shows is likely to recur.

10. Cross-Module Data Flow

The diagram below traces one business day's data through the system, from Rate Master's pricing at the top, through the two pump submissions, into the Trial Balance, and out to reports.

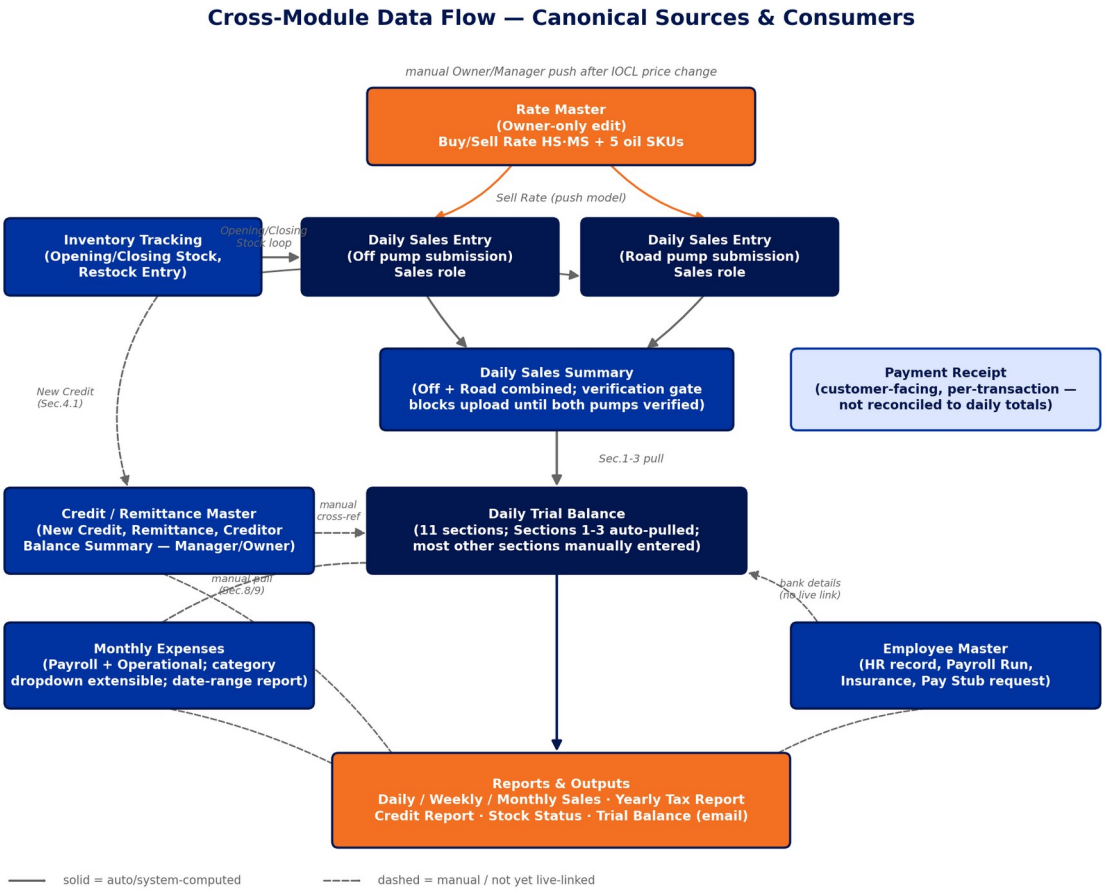


Figure 3 — Cross-module data flow. Solid arrows are system-computed/auto-pulled; dashed arrows are manual entry or cross-referenced without a live link today.

10.1 Walk-Through of a Single Day

- Pricing: the Owner (only) updates Rate Master when IOCL changes prices, then explicitly pushes the new Sell Rate to Daily Sales Entry and Trial Balance — rates are deliberately locked (not live-linked) so they never shift under a Pump Sales Man mid-entry (BRD Section 5.26).
- Entry: up to two Sales users, one per pump (Off / Road), each submit their own Daily Sales Entry — variable count, since a pump can be out of service or covered by a substitute (BRD Section 5.4).
- Combination: Daily Sales Summary combines both submissions and will not allow the upload to proceed to Trial Balance until both pumps show verified — this closes a real architectural gap the BRD itself found (Section 5.23): earlier “sync” buttons implied a direct Daily Sales Entry → Trial Balance link that never actually modeled how two independent submissions get combined into Trial Balance's 4-row Section 2/3 structure.
- Reconciliation: Trial Balance pulls Sections 1–3 from the Summary and from Inventory/Rate Master, then requires manual entry for cash, bank, and credit reconciliation sections — this manual/auto split is exactly what Section 9's formula register documents field-by-field.
- Downstream: Credit/Remittance Master, Monthly Expenses, and Employee Master each own their respective data (credits, expenses, payroll) and are cross-referenced into Trial Balance manually

today — see Section 8.5 for why this is flagged as an open architectural decision, not treated as settled.

- Output: the Daily Trial Balance Report is emailed to management daily; Daily/Weekly/Monthly Sales Reports go to the Owner and the bank; a Yearly Sales Report supports the annual tax filing (through the station's CA).

11. OCR / Upload Mode & Manual Mode Design

11.1 Pipeline

Scenario: a pump sales man fills a pre-printed paper form by hand, scans it on the attached HP printer/scanner, and uploads the image/PDF (BRD Section 5.4, Scenarios 1–3). The pipeline:

- 1. Upload — the file (JPG/PNG/PDF) is received by the Backend Service, tagged with the current Pump Serial# and Last Shift Reading context.
- 2. Extraction — Tesseract OCR (open-source, bundled in the installer, invoked on demand — not a running service) extracts field values. Tesseract is optimised for printed/typed text; accuracy on handwriting is expected to be lower and must be validated during prototyping.
- 3. Recomputation — the same calculation engine that drives manual entry (Section 7.3) re-runs against the extracted values.
- 4. Validation & flagging — any row where the OCR-read Amount does not match the recomputed value is flagged, catching both OCR misreads and inconsistent handwritten totals.
- 5. Human review — flagged rows are surfaced to the user before final save; this human-review-before-save step is a required design element for every non-manual entry path (OCR and Excel import alike), not a nice-to-have.
- 6. Save — once confirmed, the record is written with entry_mode = 'ocr' for traceability.

11.2 Test Scenario Library (from real scanned samples)

These scenarios come directly from real handwritten sample forms reviewed during the BRD phase (Section 8) and are carried forward here as the required OCR/Manual-Mode test cases, per the BRD's own instruction ("these scenarios should be carried into the System Design & Architecture document as concrete Upload Mode and Manual Mode validation test cases").

Observed Pattern	Required Handling
Multiple scratch-work lines in one Amount cell	Treat the row-level Amount as free-text/expression input; the review step must let staff confirm which line is the intended final value rather than auto-summing every line.
Hand corrections / overwrites	Manual Mode allows easy inline correction pre-save; Upload Mode (OCR) flags low-confidence or overwritten digits for human review rather than silently accepting the first reading.
Free-text values in structured-looking fields	Card Type stays free text, not a fixed dropdown; oil item names remain editable; Quantity accepts text, not just a plain number.
Orphan / non-reconciling totals	Totals auto-calculate from entered row data wherever possible; if a worker-entered total doesn't reconcile with its rows, the form flags the mismatch rather than silently storing it.
Missing key totals	Every Total / Net Bal Hand-off field is system-calculated, so a skipped manual sum never blocks or delays submission.

Observed Pattern	Required Handling
Pre-printed toggle corrected by hand	The corresponding field stays a selectable toggle/dropdown in the digital form, removing the need for manual strike-through.
Fully empty repeating sections	All repeating-row sections must submit successfully with zero rows entered.

12. External Statement Reconciliation

Three external PDF statements reconcile specific Trial Balance fields, all confirmed as native digital PDF exports (not scanned — text-based extraction applies, not OCR).

Statement	Reconciles	Confirmed Formula / Mapping
Indian Bank Daily Statement	"Bank balance INDIAN BANK" (Trial Balance Sec.4)	The account is an overdraft (OD) account. Field = OD Limit − Statement Ending DR Balance, further adjusted for a blocked/uncleared amount (the station's own "@Fraud Pending" label). OD Limit is not present in the PDF and must be configured per station. Verified example: 1,800,000 − 1,354,103.79 = 445,896.21, vs. the recorded 432,530.00 — the 13,366.21 gap is exactly the blocked-amount adjustment.
IOC Account Statement Report	"Fleet card" cleared amount (Sec.4 / Sec.11)	Direct (absolute-value) pull from the statement's Closing Balance — no formula. Verified: statement −188,501.27 vs. Trial Balance 188,501.
Yes Bank Daily Statement	"Bank balance YES BANK (QR CODE AMOUNT)" (Sec.4)	Direct pull from the statement's Running Balance (last transaction). Verified: 686.50 vs. 686.

Entry mode: both auto-fill-on-upload and manual entry/override must be supported, with manual entry as the primary, well-supported path — confirmed real-world usage is that all three fields are entered manually most of the time, with PDF upload as a secondary convenience, not the assumed default. Uploading the PDF auto-fills the corresponding field; the user reviews and can correct it before saving, consistent with the human-review-before-save principle already required for OCR (Section 11).

> *Open item: the necessity of the separate Fleet Card Balance Tracking sub-section (New/Old Airtel Balance fields) is unresolved — the IOC statement's Closing Balance already covers the Fleet card reconciliation directly, and "Airtel" was later clarified to be a creditor's name, not the telecom company (BRD §9, item 6/1). Confirm final field scope with the station before implementing Trial Balance Section 11.*

13. Security Architecture

13.1 Authentication & Password Reset

- Login identity: every user has a personal email on file; a system-generated Login Name (first initial + last name, editable on collision) is used to sign in.
- Password reset: email-confirmation link, single-use and time-limited. Two entry points — self-service "Forgot Password" from the login screen, and admin-initiated "Reset Password" per user from Manage Users (covers both a newly-created user setting their own first password, and an existing user's reset). Neither path ever displays, sets, or transmits an actual password value — no Manager, Administrator, or Owner ever sees a password.
- OTP: recommended as the verification model for login/reset, deliverable via email (SMS OTP would require a paid gateway — evaluate only if the client later requests it).

13.2 Two-Factor Authentication (TOTP)

Google Authenticator-compatible TOTP (RFC 6238), optional per user (confirmed — not mandatory), set up as a simple Enabled/Disabled field on the user record rather than a permanently-visible section (a UI structure the BRD explicitly corrected after client feedback). The mockup's client-side TOTP computation was verified byte-identical against a Python reference implementation — but this verification exists only to validate the algorithm choice.

> *CRITICAL, restated from the BRD verbatim because it is a security-load-bearing requirement: TOTP verification MUST happen server-side, in the Backend Service, never in the Electron renderer. A client-side-only check can be bypassed by a modified client reporting false success. This is not a suggestion — it is the one security requirement the BRD flags as non-negotiable.*

Open, unresolved by the client: whether an enabled account needs the code on every login or only for sensitive actions (Rate Master updates, Manage Users), and the account-recovery path if a user loses their authenticator device. Both must be settled before backend implementation (Section 19, item 6).

13.3 Sensitive Data Protection

Fields the BRD explicitly flags as requiring protection beyond the rest of the system's data, none of which is implemented yet (the mockups are demo-only and deliberately avoid fabricating realistic-looking values for these fields):

Field	Location	Requirement
Bank Account Number, IFSC Code, Bank Branch	Employee Master	Encryption at rest; role-restricted to Manager/Owner only (already access-gated at the module level, Section 4.2)
Cell Phone Number	Manage Users, Credit/Remittance Master	Flagged as comparable personal data; same handling standard as Account Number
TOTP secret	users table	Never displayed after initial setup; not the same value as the phone number, and losing the phone number alone does not compromise 2FA
External statement PDFs (Indian Bank, IOC, Yes Bank)	external_statement / uploaded files	Contain full account/statement detail; store with the same access restriction as the financial data they reconcile

13.4 Audit Trail

BRD Section 5.27 states this as a critical, non-negotiable requirement: every record in every module must capture who last saved it and when. The design generalises this as an audit_log table (Section 8.2) capturing every write, plus a last_updated_by/last_updated_at pair directly on every transactional table for fast display — the pattern already validated across all 12 active mockups via the “Logged in as” selector.

13.5 RBAC Enforcement — Defense in Depth

See Section 4.3 — UI-level hiding plus an independent server-side check on every write, never client-side-only enforcement.

14. Deployment Architecture

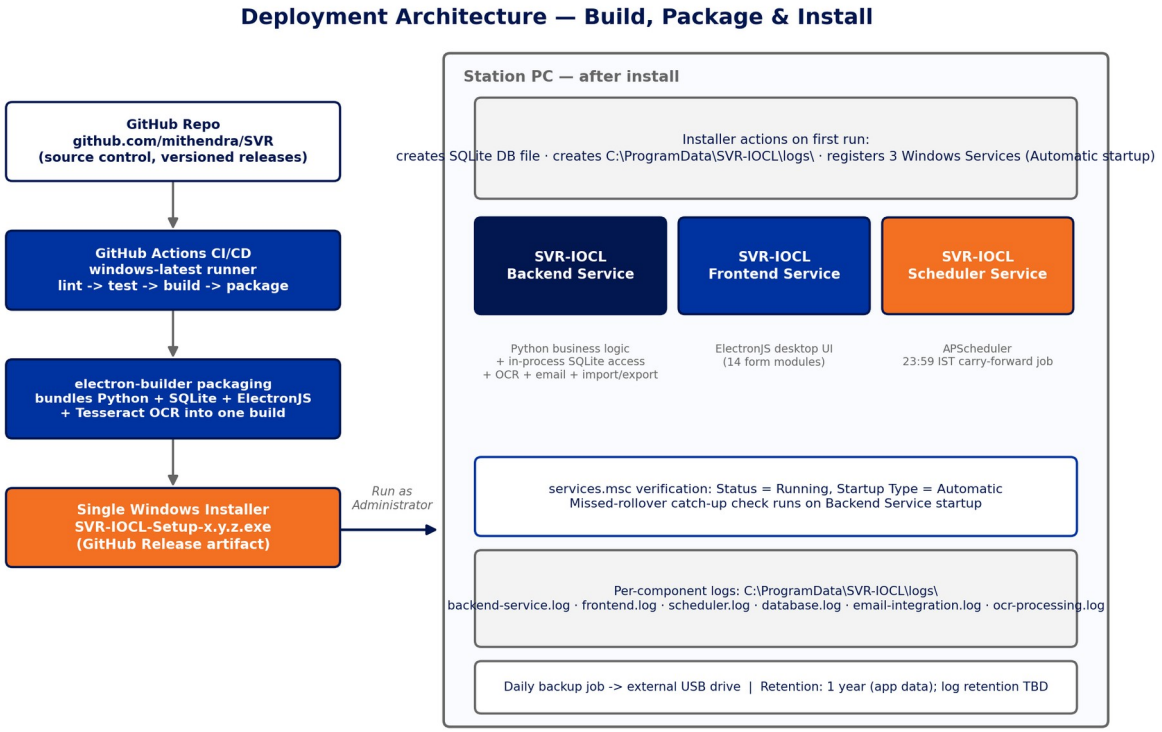


Figure 4 — Build, package, and install flow: from GitHub through CI/CD packaging to the three Windows Services running on the station PC.

14.1 Install Flow

- The end-user PC needs nothing pre-installed: a single .exe installer bundles Python, SQLite, ElectronJS, and Tesseract OCR (confirmed 2026-08-29 — Tesseract's bundling does not add meaningful complexity).
- Run the installer as Administrator — required because registering Windows Services needs elevated privileges.
- On first run: creates the SQLite database file, creates the log directory structure, and registers the three Windows Services with Automatic startup type.
- On completion, all three services should already be running — no manual launch, on this or any subsequent restart.

14.2 Verifying Services

services.msc → locate the three SVR-IOCL service entries → Properties shows Status (Running/Stopped) and Startup Type (should read Automatic). A Stopped or failed service is diagnosed via that component's own dedicated log file — never a shared/mixed log.

14.3 Log File Locations

Component	Log File (proposed)
Python backend service	C:\ProgramData\SVR-IOCL\logs\backend-service.log
SQLite / database layer	C:\ProgramData\SVR-IOCL\logs\database.log
ElectronJS frontend	C:\ProgramData\SVR-IOCL\logs\frontend.log

Component	Log File (proposed)
Scheduled job (23:59 IST)	C:\ProgramData\SVR-IOCL\logs\scheduler.log
Email integration	C:\ProgramData\SVR-IOCL\logs\email-integration.log
OCR / Tesseract processing	C:\ProgramData\SVR-IOCL\logs\ocr-processing.log

> *C:\ProgramData is used rather than a per-user folder because these are system-level Windows Services, not tied to whichever staff member is logged in — accessible regardless of which shift is on. Log retention policy (same 1-year as application data, or a shorter rotation) is an open item — Section 19, item 22.*

15. CI/CD Pipeline

GitHub Actions on the GitHub-hosted windows-latest runner (confirmed, to match the single .exe installer target platform), source-controlled at github.com/Mithendra/IOCL_SVR.

- Lint & test on push — pytest for the Python backend; Playwright for the ElectronJS UI, exercising the same calculation-chain and RBAC-gate assertions already validated across every mockup during the BRD phase.
- CER checks (Correctness, Efficiency, Reliability — the BRD's own stated standard, Section 1): automated tests for data-entry validation, database read/write integrity, and service auto-restart behaviour.
- Package — electron-builder produces the single Windows .exe, bundling Python + SQLite + ElectronJS + Tesseract.
- Release — versioned GitHub Releases; the installer artifact is what Section 14's install flow consumes.

Development-process recommendation carried from the BRD (Section 4.7, sourced from Anthropic's own Claude Code documentation, given the number of shared formula dependencies discovered across this project — e.g. a change to Section 1's testing deduction affects Margin, Margin Total, and Total Sale Amt several sections away): default to Plan Mode (a read-only review-before-execution mode) for any change touching more than 2–3 files or a shared formula, and maintain a single project-level CLAUDE.md capturing the conventions this project already leans on — the Sell-Rate-vs-Buy-Rate distinction, the 23:59 IST carry-forward mechanism, the human-review-before-save pattern, and the standing discipline of verifying every formula against real source data before calling it correct.

16. Non-Functional Requirements

16.1 Performance & Scale

Confirmed as a small, single-station dataset — a handful of records per shift, up to two per day per pump. Load-tested during the BRD phase at realistic-to-generous volumes with no issues: the 26-column ledger at a full year's rows (365, added in 0.32s, calculated in 0.008s), and oil-item extensibility at 50 items. No special scaling or archival infrastructure is required.

16.2 Reliability

- All three Windows Services auto-start on machine startup — no manual launch, ever, including after a crash or power loss.
- Per-component logging (Section 14.3) so a failed component is diagnosable without digging through a shared log.

- The 23:59 IST carry-forward job has an explicit catch-up-on-startup requirement for a missed rollover (Section 7.7) — reliability here is a named design requirement, not an assumption.

16.3 Offline-First, Single-Station Deployment

No cloud dependency in the core system — the station operates fully offline except for the optional email integration (report delivery, password reset) and future PDF-statement upload convenience. Cross-browser testing is confirmed not needed: the ElectronJS desktop architecture does not run in a general-purpose browser in production.

16.4 Localization

Static, pre-translated English/Telugu labels for section headings, switched instantly client-side — no external translation service dependency, confirmed as a deliberate design choice to avoid runtime translation-API cost or latency for a UI this small.

16.5 Print & Paper Standards

A4 Landscape is the final, production-confirmed orientation for Daily Sales Entry. This reversed twice during the BRD (Landscape → Portrait, on the reasoning that the original paper form is A4 Portrait, 1 page → back to Landscape, on direct client confirmation that Landscape is what is actually in production, printing correctly on one page today). The lesson recorded in the BRD alongside this decision is worth repeating here as a working principle for the design/build teams: written documentation of a "confirmed, corrected" decision is not automatically more trustworthy than the current working code, especially on a project this long-running — when the two disagree, check with the client rather than assume either source is right by default. Legal/Foolscap paper was researched and explicitly rejected as a fallback (not commonly stocked in India, and a real naming trap where "legal size" locally often means Foolscap, not US Legal).

16.6 Auditability

Every table carries a last-updated-by/date pair and a corresponding audit_log entry (Section 13.4) — stated by the client as a critical, non-negotiable requirement, not a nice-to-have.

17. Testing Strategy

Formal test cases are required (BRD Section 4.5), covering both automated (CI/CD) and manual/scenario-based testing. Recommended tooling — pytest for the backend, Playwright for the UI — is a low-friction default given the project's existing familiarity: Playwright is the exact tool used to validate the calculation engine, formula chains, and RBAC gates across all 14 mockups throughout the BRD phase.

Category	Coverage
OCR / Upload Mode	The 7 real handwriting scenarios catalogued in Section 11.2, converted into repeatable automated test cases against the Tesseract pipeline
Service reliability	Each of the 3 Windows Services auto-starts correctly on startup; a forced failure of each component correctly writes to its own dedicated log file
Data entry / reconciliation	Daily Sales Entry, Credit/Remittance Master, and Daily Trial Balance field validation — including the Indian Bank OD formula (Section 12) and the full formula register (Section 9)
Report / email	The Daily Trial Balance Report and the Daily/Weekly/Monthly/Yearly Sales Reports generate correctly and, where applicable, send via email as designed

Cross-browser testing is confirmed not required (Section 16.3). Whether test cases live in the GitHub repo alongside the CI/CD pipeline is assumed yes, consistent with Section 15, and should be confirmed as a formality rather than treated as open.

18. Architecture Decision Records

ADR-1 — Canonical source-of-truth entry points, with Trial Balance as a rollup (proposed, not yet adopted)

Decision: Recommend converting Daily Trial Balance's Sections 3, 8, and 9 from manually-entered duplicate fields into read-only rollups computed from Credit/Remittance Master, Monthly Expenses, and Employee Master's Payroll Run respectively. Status: RECOMMENDED, EXPLICITLY DECLINED BY CLIENT DURING THE BRD PHASE — kept as a design option pending revisit, per Section 8.5.

Rationale: A shared database does not by itself prevent the same real-world transaction from being typed into two forms. This decision was raised, discussed, and consciously deferred rather than resolved — it should be re-confirmed before backend development locks in Trial Balance's schema, since building manual columns vs. computed views are materially different engineering paths to unwind later.

ADR-2 — Three Windows Services, not four or five

Decision: Backend, Frontend, and Scheduler are separate Windows Services. SQLite does not get its own service (it is a file, accessed in-process). Tesseract does not get its own service (it is a library invoked on demand).

Rationale: A 4th “SQLite service” and a separate “Tesseract service” were explicitly proposed and rejected as technically unnecessary — consistent with the client's own direction to keep this small-footprint (~10 user) application simple rather than over-engineered, the same principle that later collapsed the role model from 4 roles (briefly 4, with Read Only) back to 3.

ADR-3 — Versioned, database-backed business-rule parameters

Decision: Store frequently-revised constants (the Section 1 testing/density deduction, the ±Rs100 alert threshold, the yet-to-be-confirmed daily allowance) in a system_parameter table with effective dates and an updated_by audit trail, rather than hardcoding them.

Rationale: The testing deduction alone was corrected 5 times across 3 sessions (10.0 → 10.5 → 21 (a documented misreading) → 10.5 → 10) and the Stock Value rate was corrected 3 times in a single day. This is the project's own demonstrated failure mode; the fix is architectural, not procedural.

ADR-4 — Server-side-only TOTP verification

Decision: 2FA code verification happens exclusively in the Backend Service. The Electron renderer never performs the comparison.

Rationale: Explicitly flagged in the BRD as a critical security requirement — a client-side-only check can be bypassed by a modified client reporting false success.

ADR-5 — Human-review-before-save on every non-manual entry path

Decision: OCR-extracted and Excel-imported records are recomputed by the calculation engine and any mismatch against the read value is surfaced to the user before final save — on every entry path, not just manual entry.

Rationale: This is the confirmed design for OCR (BRD Section 5.4) and generalises directly to Excel import, which converges on the same underlying record structure.

ADR-6 — Single .exe installer bundling every component, including Tesseract

Decision: One Windows installer, built via electron-builder, bundles Python, SQLite, ElectronJS, and Tesseract OCR — the station PC needs no separate prerequisite installation.

Rationale: Confirmed default/recommended approach from early in the BRD, reconfirmed 2026-08-29 specifically for Tesseract's bundling — this does not add meaningful complexity and means Scan/Upload works immediately after installation with no separate setup step.

19. Open Design Items Carried Into Development

Every item below was explicitly flagged as PENDING, OPEN, or UNDER REVIEW somewhere in the BRD's 105-session history, or is a design-phase question this document itself surfaces. None are assumed resolved. This table is the single consolidated punch list for the client and development team before implementation begins — resolved items are included too, marked RESOLVED, so the register is complete rather than only showing what's left.

#	Open Item	BRD Ref	Status / Recommendation
1	Fleet Card Balance Tracking (New/Old Airtel Balance) necessity	§4.4 / §5.8 Sec.11	Confirm final Trial Balance Section 11 field set with the station — the IOC statement's Closing Balance may already cover this reconciliation
2	Phone Pay Settled timing (6:30 AM IST assumption)	§4.4	PENDING — confirm directly with the station before implementing the settled/unsettled split
3	Excel export “same format” fidelity	§5.4.3 / §5.5.1	RESOLVED 2026-08-29 — structured data export confirmed sufficient, no paper-form visual replication required
4	Section 10 (Daily Mgmt Summary) dedicated export target format	§5.5.1	Decide single-row-daily vs. running multi-day table, plus file-naming convention
5	Theme preference scope: global vs. per-form	§5.12	Confirm before designing user_preference.scope in the schema
6	2FA login frequency & account-recovery path if device lost	§5.14	Confirm before implementing TOTP enforcement policy server-side
7	Rate Master effective-dating & rate versioning for historical records	§5.11	Confirm whether past Daily Sales/Trial Balance records should freeze the rate active at creation time
8	Inventory Reorder Level thresholds + low-stock notification channel	§5.10	Confirm thresholds per SKU with the station; decide in-app vs. email notification
9	Stock Status Report — standalone report or not	§5.10 / §6	Confirm with the station
10	Old Credit running-balance/aging view, partial-repayment rules	§5.9	RESOLVED — partial repayments confirmed as the norm (2026-08-28); Credit/Remittance Master Sec.3 implements the aggregation
11	Section 1 “IOCL Adv” column formula	§5.8.5	Insufficient data to derive reliably from a single day — needs more source data or direct client confirmation
12	“Actual Profit After All Expenses ~Rs3,300” formula (Row 99)	§5.8.10 / §5.9	Verified only in the client's own Excel — implement in the digital form once confirmed as canonical
13	Section 8 dual-column (C vs D) source ambiguity	§5.8.5	RESOLVED 2026-08-17 — column D confirmed not needed

#	Open Item	BRD Ref	Status / Recommendation
14	“GOOD”/“Not Good” status auto-text	§5.8.5 / §5.8.10	Threshold rule confirmed (±Rs100, the client's own wording) — display logic still manual entry, not yet auto-generated
15	Buy Rate margin/profit calculation not consumed anywhere	§5.11	Open design question — should Buy Rate feed a margin calc in Trial Balance §7/§9?
16	Section 11 (Old/New Credit Sales Details) scope	§5.8.5 / §5.8.6	UNDER REVIEW — only 2 rows observed in source data; needs more days of data or direct scoping
17	Employee self-service access model	§5.16	RESOLVED 2026-08-28 — no employee-facing access; Manager/Supervisor provides pay-stub info informally
18	Data normalization / canonical source-of-truth (Trial Balance duplication)	§5.8.8 discussion	Recommendation given, explicitly declined by client for now — see ADR-1; revisit before Trial Balance schema is finalised
19	“Logged in as” demo selector → real session-based authentication	§5.27, all forms	Must be replaced by real login (§5.1) and server-side session/role checks before production — see §13.5
20	Sensitive field encryption (Account Number, IFSC, phone numbers, TOTP secret)	§5.16 / §5.17	Required before production use; not implemented in the mockups by design — see §13.3
21	Payment Receipt — no language toggle	§5.30	RESOLVED 2026-08-27 — confirmed English-only, a deliberate choice
22	Log retention policy (1-year, same as app data, or shorter)	§4.6	Confirm with the station
23	Exact Windows Service names / SQLite file path / Electron service-vs-startup-item	§4.6 / §4.8	3 services named and largely finalised; confirm whether ElectronJS truly runs as a Windows Service or a user-session startup item
24	RBAC re-validation of remaining modules under the final 3-role model	§2.1	Several modules were gated under the earlier 4-role naming and not explicitly re-tested — close before development sign-off; see §4.2 matrix
25	Where the retired Administrator role's permissions land	§2 / §2.1	This document assumes they fold into Manager (already a superset) — confirm with the client before development

20. Appendices

Appendix A — Glossary

Term	Meaning
HS / MS	High Speed Diesel (HS) / Motor Spirit — Petrol (MS), IOCL's standard fuel-grade abbreviations
IOCL	Indian Oil Corporation Limited — the parent brand; SVR is a branded dealership station
Nz1 / Nz2	Nozzle 1 / Nozzle 2 — physical dispenser nozzle identifiers, appended to fuel rows on later form revisions
Off / Road (pump)	The station's two physical dispensing locations — “Office Near by Pump 1” and “Road Side Pump 2” in full, shortened variously to Off/Road, Off Front/Road Front across the BRD's history

Term	Meaning
OD (account)	Overdraft — the Indian Bank account type reconciled in Trial Balance Section 4
TOTP	Time-based One-Time Password (RFC 6238) — the 2FA mechanism used in Manage Users
Carry-forward	The 23:59 IST scheduled job copying each day's Current Reading into the next day's Last Shift Reading
CRAFT	Context, Role, Action, Format, Target — the BRD's own documentation framework, referenced in this document's section numbering lineage

Appendix B — Legacy Role → Final Role Mapping

Original (pre-2026-08-29)	Final (confirmed)
Data Entry (Pump Sales Man)	Sales
Manager	Manager (unchanged)
Super User (Dealership Owner)	Owner
Administrator	Assumed folded into Manager — NOT explicitly confirmed (Section 19, item 25)
Read Only (introduced and eliminated same day, 2026-08-29)	Removed — not part of the final model

Appendix C — Source Package Inventory

Files supplied in the BRD package and their role in producing this document:

- SVR-BRD-Requirement-Gathering.docx — the current-state BRD (primary source for Sections 3–14 of this document)
- SVR-BRD-Session-Update-2026-08-04.md — 105-session chronological log (primary source for Section 9's formula register and Section 19's open items)
- 14 branded HTML mockups — reference implementations for Sections 5 and 6
- SVR-Software-Install-Guide.pdf — source for Section 14's install flow
- GAS_STATION_AUG11_AUG12.xlsx and 3 SVR_Gas_Station_Daily_Sales_Report *_Filled workbooks — the real data every formula in Section 9 was verified against